Building a Real-Time Chat and Voice Platform: A Deep Dive into Discord-Scale System Design

Blog Details

- **Author**: Naveen R.
- **Date**: September 2, 2026
- **Tags**: distributed systems, real-time communication, system design, WebSockets, WebRTC
- **Read Time**: 20 mins

When I sit down to design a real-time communication platform like Discord (with its servers, channels, text chat, and voice/video huddles), I'm immediately confronted with challenges that dwarf most typical web applications. We're talking about pushing messages to millions of concurrent users in milliseconds, storing trillions of messages efficiently, routing high-quality audio and video streams globally, and broadcasting presence updates without melting our infrastructure. This isn't just a CRUD app with WebSockets bolted on; it's a distributed systems problem that touches every corner of backend engineering.

In this post, I'll walk through exactly how I would design such a system from the ground up. I'll start by scoping the problem and establishing requirements, then build out a high-level architecture before diving deep into four critical subsystems: the WebSocket gateway for real-time messaging, message storage at massive scale, voice and video infrastructure, and presence/state management. Throughout, I'll anchor on Discord's architecture (servers/guilds, channels, text and voice) while calling out how platforms like Slack and Microsoft Teams differ in their approach.

Scoping the Problem and Clarifying Assumptions

Before I write a single line of code or draw an architecture diagram, I need to understand exactly what I'm building. Let me break down the core features and establish some boundaries.

Core Features

Server/Guild Structure:

- Users create or join servers (Discord calls them "guilds" internally)
- Each server contains multiple text and voice channels
- Servers can have thousands to hundreds of thousands of members
- Role-based permissions control who can see/access what

Text Chat:

- Real-time message delivery to all channel members
- Message history and pagination
- Support for text, images, videos, and rich embeds
- Message editing and deletion

Voice and Video Huddles:

- Multiple users can join a voice channel simultaneously
- Optional video streaming
- Screen sharing capabilities
- Low-latency audio (sub-150ms for natural conversation)

Presence and Status:

- Online/offline/idle/do-not-disturb status
- "Typing..." indicators in text channels
- Voice channel occupancy (who's in which voice room)

Key Assumptions

For this design, I'm assuming:

1. **Scale similar to Discord's public metrics:** ~50 million daily active users, with ~10 million concurrent online users at peak
2. **Message volume:** Approximately 1 billion messages per day
3. **Voice usage:** ~2.5 million concurrent voice users
4. **Global distribution:** Users spread across multiple continents, requiring regional infrastructure
5. **Availability target:** 99.99% uptime (critical for always-on gaming communities)
6. **Data retention:** Messages stored indefinitely unless explicitly deleted

How Slack and Teams Differ

Slack organizes around workspaces rather than servers, typically with smaller member counts (dozens to hundreds rather than thousands). This changes the scaling profile: fewer massive fan-outs, but more isolated workspaces to manage.

Microsoft Teams integrates tightly with Office 365 and emphasizes enterprise features like compliance, e-discovery, and Active Directory integration. Their voice/video infrastructure leverages existing Skype for Business technology and focuses heavily on meeting rooms rather than persistent voice channels.

Discord's unique challenge is handling massive public communities (servers with 100,000+ members) alongside smaller friend groups, all on the same platform.

Functional and Non-Functional Requirements

Functional Requirements

1. **User Management:**
    - Users can create accounts, join multiple servers
    - Friend connections between users across servers
2. **Server/Channel Management:**
    - Create servers with text and voice channels
    - Invite users via links or direct invites
    - Role-based permissions
3. **Real-Time Messaging:**
    - Send text messages to channels
    - Receive messages in real-time (< 1 second delivery)
    - View message history with pagination
    - Edit and delete messages
4. **Voice/Video Communication:**
    - Join voice channels
    - Stream audio with low latency (< 150ms)
    - Optional video streaming
    - Screen sharing
5. **Presence:**

- See online/offline status of friends
- See who's in voice channels
- Typing indicators in text channels

Non-Functional Requirements

1. **Low Latency:**
   - Message delivery: < 1 second
   - Voice latency: < 150ms (critical for natural conversation)
   - Typing indicators: < 500ms
2. **High Availability:**
   - 99.99% uptime target
   - Graceful degradation when components fail
3. **Scalability:**
   - Support 10 million concurrent connections
   - Handle 60,000 peak messages per second
   - 2.5 million concurrent voice users
4. **Consistency:**
   - Strong consistency for message ordering within a channel
   - Eventual consistency acceptable for presence updates

Back-of-the-Envelope Capacity Estimation

Let me work through the numbers to understand what infrastructure I'll need.

Daily Active Users (DAU): 50 million
Concurrent Users: 10 million (20% of DAU online simultaneously)

Message Traffic:

- **Daily messages:** 1 billion
- **Average QPS:** 1,000,000,000 / 86,400 seconds ≈ **12,000 messages/sec**
- **Peak QPS (5x average): 60,000 messages/sec**

Storage Requirements:

- **Average message size:** 100 bytes (text + metadata)

- **Daily storage:** 1 billion × 100 bytes = **100 GB/day**
- **Yearly storage:** 100 GB × 365 = **36.5 TB/year**
- **Media attachments (10% of messages at 500KB avg):** 100 million × 500KB = **50 TB/day**

WebSocket Connections:

- **Concurrent connections:** 10 million
- **Connections per Gateway server:** ~100,000 (assuming 16-core machines with optimized networking)
- **Gateway servers needed:** 10,000,000 / 100,000 = **100 servers minimum**
- **With redundancy (2x): 200 Gateway servers**

Voice Infrastructure:

- **Concurrent voice users:** 2.5 million
- **Average voice channel size:** 5 users
- **Active voice channels:** 2,500,000 / 5 = **500,000 channels**
- **Media servers needed:** Depends on regional distribution and capacity per server

Database Sizing:

- **Message database:** Starting at ~40 TB, growing 36 TB/year
- **With replication factor of 3: 120 TB initial capacity**
- **Distributed across cluster:** If using ScyllaDB-style architecture, 72 nodes (Discord's actual count post-migration) with ~1.7 TB per node

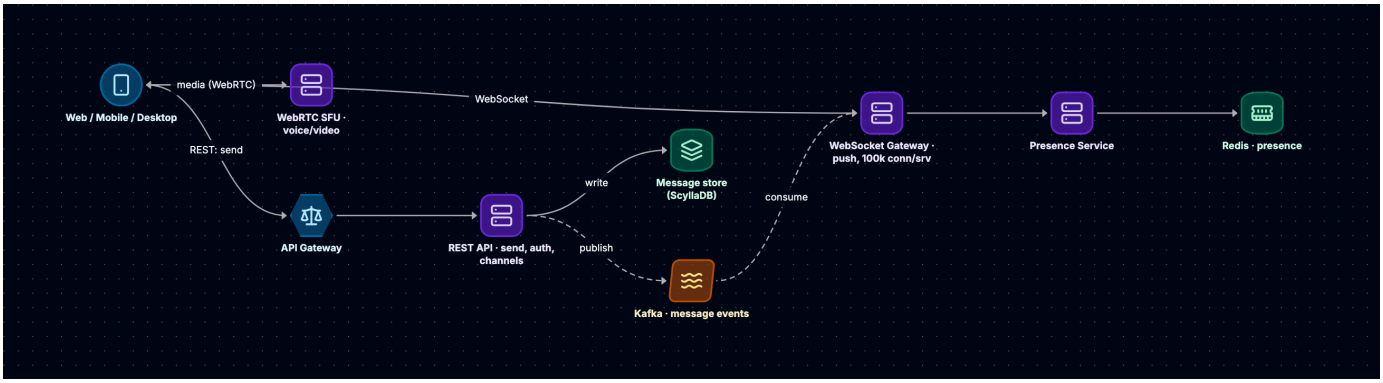
Bandwidth:

- **Average message size:** 1 KB (including protocol overhead)
- **Outbound bandwidth for messages:** 60,000 msg/sec × 1 KB = **60 MB/sec = 480 Mbps** (just for message content)
- **Fan-out multiplier:** In a 100-member channel, 1 message becomes 100 outbound pushes
- **Realistic peak outbound:** Several **Gbps** across all Gateway servers

These numbers tell me I need a highly distributed architecture with aggressive caching, request coalescing, and careful attention to hot spots.

High-Level Architecture and Core Components

Here's how I would structure the system at a high level:



Component Breakdown

API Gateway / Load Balancer:

- Routes incoming HTTP and WebSocket requests
- Returns optimal WebSocket Gateway server URL based on user location
- SSL termination

REST API Servers (Stateless):

- Handle message sending: `POST /channels/{channel_id}/messages`
- Message history: `GET /channels/{channel_id}/messages?limit=50&before={message_id}`
- User authentication, server/channel management
- Horizontally scalable, with no session state stored locally

WebSocket Gateway (Stateful):

- Maintains persistent connections with clients (~100k per server)
- Pushes real-time events: `MESSAGE_CREATE`, `PRESENCE_UPDATE`, `TYPING_START`
- Handles heartbeat protocol (clients ping every 40 seconds)
- Single responsibility: deliver data to connected users

Message Queue (Kafka):

- Decouples API layer from Gateway layer
- When a user sends a message via REST API, it's published to Kafka
- Gateway servers consume from Kafka and fan out to WebSocket clients
- Provides buffering during traffic spikes

Request Coalescing Layer (Rust):

- Sits between Gateway and database

- Routes requests for the same channel to the same service instance
- Merges overlapping read requests into single database queries
- Critical for handling "hot" channels with thousands of simultaneous readers

Message Storage (ScyllaDB/Cassandra):

- Stores billions of messages
- Partitioned by `(channel_id, bucket)` for distribution
- Clustered by `message_id DESC` for efficient recent-message reads
- Replication factor of 3 with quorum reads/writes

Presence Service:

- Tracks online/offline/idle status
- Manages typing indicators
- Broadcasts presence updates to friends and server members

Voice/Video Infrastructure (WebRTC SFU):

- Selective Forwarding Unit servers in multiple regions
- Forwards media streams without transcoding
- Handles 2.5 million concurrent voice users

Request Flow Examples

Sending a Text Message:

1. Client sends `POST /channels/123/messages` with message content
2. REST API server validates, generates message ID (Snowflake)
3. Message written to ScyllaDB
4. Message published to Kafka topic
5. Gateway servers subscribed to that channel consume from Kafka
6. Gateway servers fan out to all connected clients in that channel via WebSocket

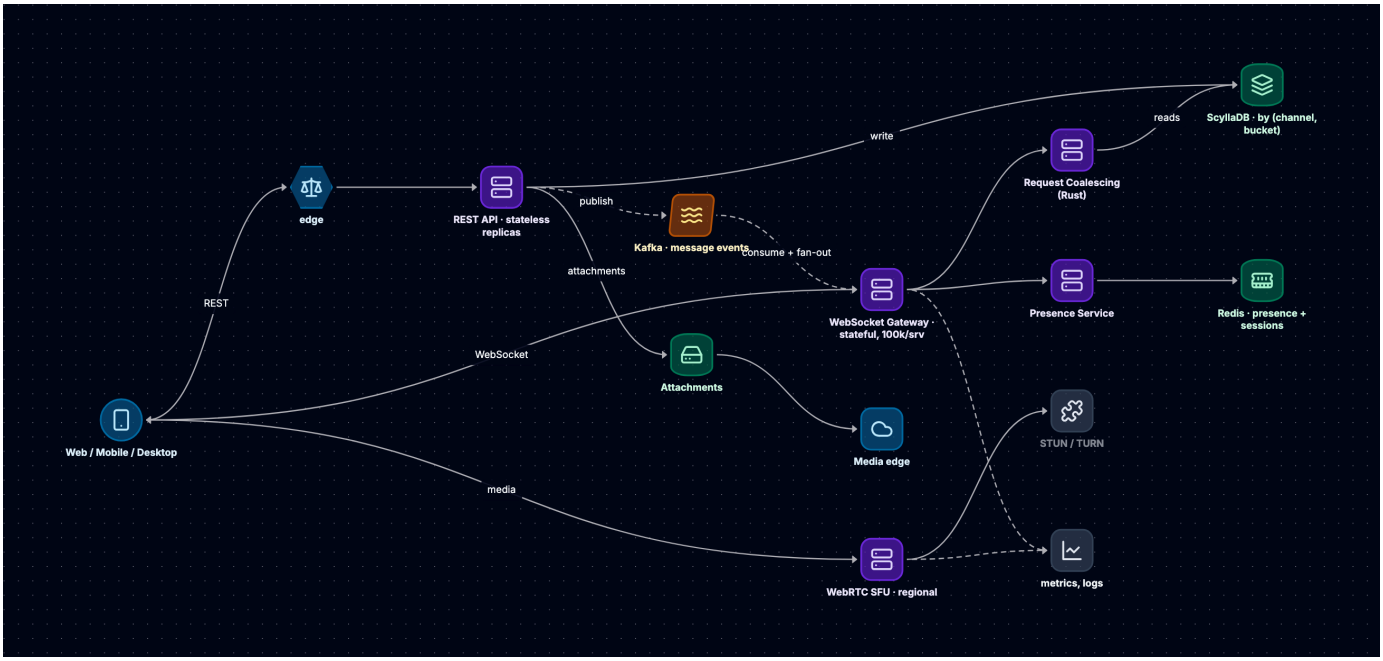
Joining a Voice Channel:

1. Client sends `POST /channels/456/join-voice`
2. API server finds optimal SFU media server (regional, load-based)
3. Returns WebRTC connection info (ICE candidates, STUN/TURN servers)
4. Client establishes WebRTC connection directly to SFU

5. SFU forwards audio/video streams to other participants

Scaling It Out

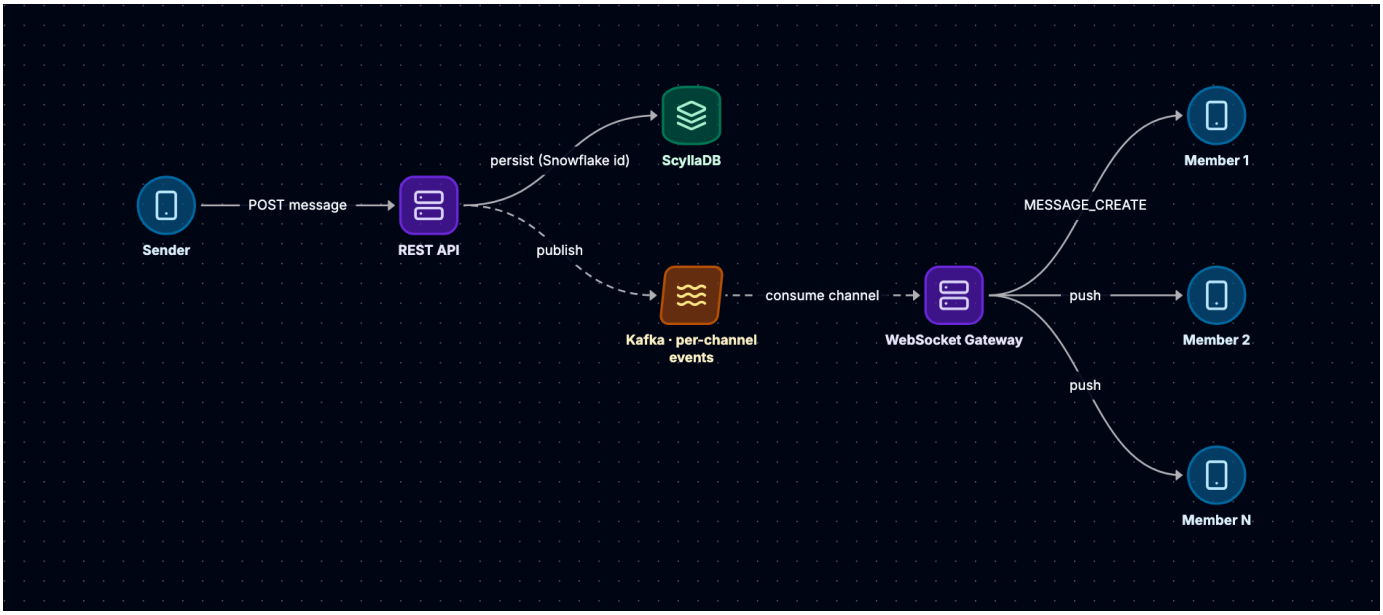
The clean view hides what it takes to hold 10M concurrent connections and 60K peak messages/second. Scaled out, I'd split the stateless REST tier from the stateful WebSocket Gateway fleet (each holding ~100K connections), keep the send path async through Kafka, put a Rust request-coalescing layer in front of ScyllaDB to survive hot channels, run presence on Redis, place SFU media servers regionally with STUN/TURN, and serve attachments from a blob store behind a CDN.



Now let's dive deep into each critical subsystem.

Deep-Dive: Real-Time Messaging and the WebSocket Gateway

The WebSocket Gateway is the beating heart of real-time communication. It's where millions of persistent connections terminate, and where every message, presence update, and typing indicator flows to users. Let me walk through how I would design this layer to handle the scale and latency requirements.



Why WebSockets Over Alternatives

For a real-time platform, I need bidirectional communication with minimal overhead. Here are my options:

- **HTTP Long Polling:** Client repeatedly requests updates. High latency (seconds), massive overhead from connection setup/teardown, inefficient.
- **Server-Sent Events (SSE):** Unidirectional (server → client only). I need clients to send data too, so I'd need a separate HTTP channel for uploads. Awkward.
- **WebSockets:** Persistent bidirectional TCP connection with minimal framing overhead. Perfect for chat.

WebSockets win for this use case. After the initial HTTP upgrade handshake, I have a long-lived connection with ~2 bytes of framing per message.

Gateway Architecture: Stateful vs. Stateless Separation

A critical design decision: I'm separating stateful Gateway servers from stateless API servers.

WebSocket Gateway Servers (Stateful):

- Each server holds ~100,000 open TCP connections
- Maintains in-memory mapping: `user_id → WebSocket connection`
- Single job: push data to users
- Cannot be arbitrarily load-balanced: once a user connects, they stick to that server until disconnect

REST API Servers (Stateless):

- Handle business logic: authentication, authorization, message validation
- Store nothing in memory about connections
- Can be freely load-balanced and scaled

Why separate them? Stateful servers are hard to scale and deploy. If I need to restart a Gateway server, 100,000 users disconnect and reconnect. By keeping business logic in stateless API servers, I can deploy updates there without affecting connections. The Gateway servers change infrequently, because they're just dumb pipes.

Connection Establishment Flow

When a user opens Discord:

1. **Client requests Gateway URL:**

   ```
   POST /gateway/url
   Response: { "url": "wss://gateway-us-east-1a.discord.gg" }
   ```

   The API returns the best Gateway server based on geography and current load.

2. **WebSocket handshake:**

   ```
   Client → Server: HTTP Upgrade request
   Server → Client: 101 Switching Protocols
   ```

3. **Authentication:**

   ```
   Client → Server: { "op": 2, "d": { "token": "...", "intents": 513 } }
   ```

   Client sends `IDENTIFY` opcode with auth token. Gateway validates with auth service.

4. **Heartbeat protocol:**

   ```
   Server → Client: { "op": 10, "d": { "heartbeat_interval": 41250 } }
   Client → Server: { "op": 1, "d": null }  (every 41.25 seconds)
   ```

   Client must send heartbeat pings. If Gateway doesn't receive heartbeat within interval + grace period, it closes the connection (assumes client crashed or network died).

5. **Initial state sync:**
   Gateway pushes current state: servers user is in, online friends, unread messages, etc.

Now the connection is live and ready to receive real-time events.

Message Delivery: From Send to Fan-Out

Let me trace a message from the moment a user hits "send" to when it appears on everyone's screen.

Step 1: Client sends message via REST API

```
POST /channels/789/messages
Body: { "content": "Hello world!" }
```

Why REST instead of WebSocket for sending? Separation of concerns. The API server handles validation, rate limiting, permission checks, and persistence. The Gateway just pushes.

Step 2: API server processes the message

- Validates user has permission to send in this channel
- Checks rate limits (prevents spam)
- Generates unique message ID (Snowflake: timestamp + worker ID + sequence)
- Writes to ScyllaDB:

```
INSERT INTO messages (channel_id, bucket, message_id, author_id,
content)
VALUES (789, 42, 123456789, 1001, 'Hello world!');
```

Step 3: Publish to Kafka

```
{
  "topic": "messages",
  "key": "channel:789",
  "value": {
    "channel_id": 789,
    "message_id": 123456789,
    "author_id": 1001,
    "content": "Hello world!",
    "timestamp": "2024-01-15T10:30:00Z"
  }
}
```

Using `channel_id` as the Kafka partition key ensures all messages for a channel go to the same partition, maintaining order.

Step 4: Gateway servers consume from Kafka

Each Gateway server subscribes to Kafka topics. When a message arrives, the Gateway:

1. Looks up which users are in channel 789 and connected to *this* Gateway server
2. For each connected user, pushes the message over their WebSocket:

```
{
  "op": 0,
  "t": "MESSAGE_CREATE",
  "d": {
    "channel_id": 789,
    "message_id": 123456789,
    "author": { "id": 1001, "username": "alice" },
    "content": "Hello world!",
    "timestamp": "2024-01-15T10:30:00Z"
  }
}
```

Step 5: Client receives and displays

The client's WebSocket listener receives the event, updates the UI, and the message appears instantly.

The Fan-Out Challenge

Here's where it gets interesting. If channel 789 has 10,000 members online across 100 Gateway servers:

- 1 message sent = 1 Kafka message published
- 100 Gateway servers each consume that message
- Each Gateway looks up local users in that channel
- If users are evenly distributed: 100 servers × 100 local users each = **10,000 WebSocket pushes**

This is the fan-out problem: One write becomes thousands of reads and pushes.

For a channel with 100,000 members (Discord has servers this large), we're talking 100,000 outbound WebSocket messages for every single chat message. At 60,000 incoming messages per second across the platform, with an average fan-out of even just 10 users per channel, that's **600,000 outbound messages per second**.

Handling Fan-Out at Scale

1. Efficient serialization:

I serialize the message payload once per Gateway server, not once per user. All users in the same channel receive identical JSON. So I:

- Deserialize from Kafka once
- Serialize to WebSocket frame format once
- Share the byte buffer across all outbound connections

2. Non-blocking sends:

I cannot let a single slow client block others. If user A is on a degraded mobile connection and can't receive data quickly, I don't want user B's message delivery to wait.

Solution: Async I/O with per-connection send buffers. If a buffer fills up (slow consumer), I either:

- Drop that connection (harsh but protects the system)
- Drop messages for that user (with a "you're behind" notification)

3. Presence-aware fan-out:

I only send messages to users who are actually online and in the channel. The Gateway maintains:

```
channel_id → Set<user_id>  (users in this channel)
user_id → WebSocket connection
```

When a user joins a channel, they're added to the set. When they leave or disconnect, they're removed. This keeps fan-out minimal.

4. Read receipts and acknowledgments:

Clients send ACKs back for received messages:

```
{ "op": 11, "d": { "channel_id": 789, "last_message_id": 123456789 } }
```

This lets the Gateway know the client is keeping up. If ACKs stop arriving, the Gateway can detect a problem.

Handling Gateway Failures

Gateways will fail. Servers crash, networks partition, deployments happen. My design needs to handle this gracefully.

Client reconnection logic:

- Clients implement exponential backoff: reconnect after 1s, then 2s, 4s, 8s, up to 60s
- On reconnect, client sends last received message ID
- Gateway sends any missed messages (if still in cache) or tells client to fetch from API

Gateway session state:

I don't store session state in the Gateway itself. Instead:

- When a Gateway receives an `IDENTIFY`, it validates the token with the auth service
- Auth service returns user ID, permissions, servers/channels they're in
- Gateway caches this for the session but treats it as ephemeral

If a Gateway crashes, users reconnect to a different Gateway, re-identify, and continue. No state is lost because the source of truth is the database and Kafka.

Monitoring Gateway Health

Critical metrics I'd track:

- **Connection count per Gateway:** Should be balanced across servers
- **Message delivery latency:** Time from Kafka consumption to WebSocket send
- **Slow consumer count:** How many connections are falling behind
- **Heartbeat miss rate:** Indicates network issues or client crashes
- **Reconnection rate:** Spike indicates Gateway instability

If delivery latency spikes above 500ms, I know there's a problem: either the Gateway is overloaded or Kafka is lagging.

How Slack and Teams Differ

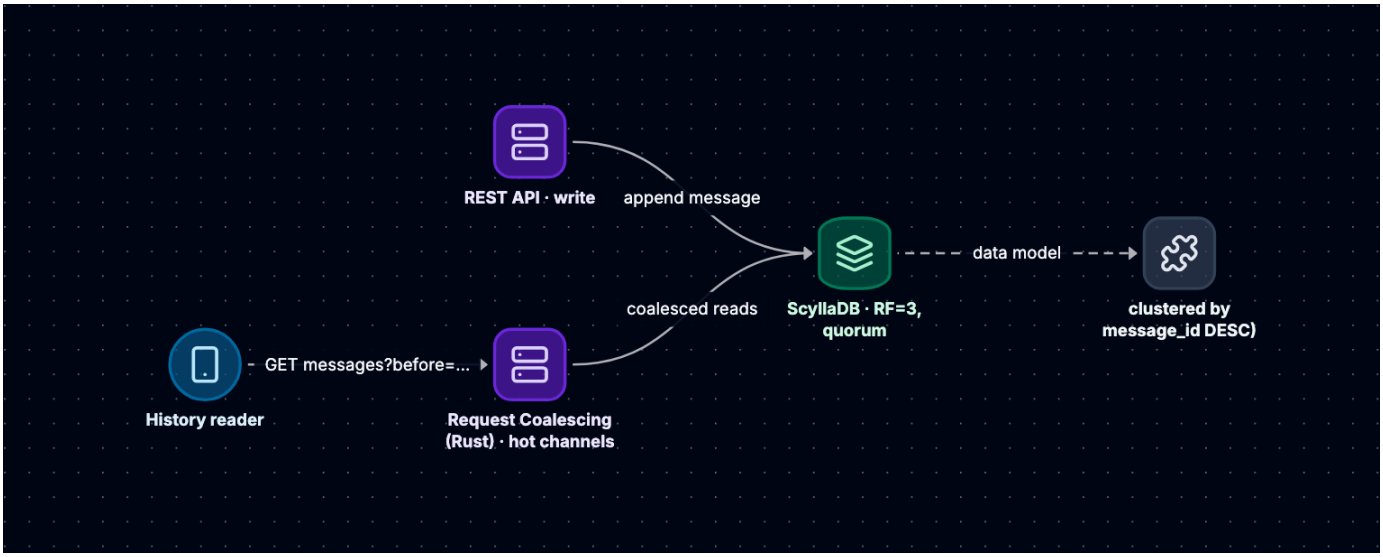
Slack uses a similar WebSocket Gateway architecture but with smaller fan-out (typical Slack channels have 10-50 members, not 10,000). They also use a "lazy loading" model where not all servers are loaded on connect, only the active workspace.

Microsoft Teams leverages Azure SignalR Service, a managed WebSocket infrastructure. This offloads the complexity of managing stateful servers but reduces control over optimization. Teams also batches presence updates more aggressively (updates every 5 minutes vs. near-real-time) to reduce fan-out load.

Discord's unique challenge is supporting both small friend groups and massive public servers on the same infrastructure, requiring more sophisticated fan-out handling.

Deep-Dive: Message Storage at Scale

Storing billions of messages efficiently is a deceptively hard problem. It's not just about disk space. It's about read latency, write throughput, hot partitions, and graceful scaling. Let me walk through how I would design the message storage layer.



Why Not a Traditional Relational Database?

My first instinct might be to use PostgreSQL with a schema like:

```
CREATE TABLE messages (
  message_id BIGINT PRIMARY KEY,
  channel_id BIGINT,
  author_id BIGINT,
  content TEXT,
  timestamp TIMESTAMP
);
CREATE INDEX ON messages(channel_id, timestamp DESC);
```

This works... until it doesn't. Problems:

1. **Sharding is painful:** PostgreSQL doesn't shard horizontally out of the box. I'd need something like Citus or manual shard management.
2. **Hot partition problem:** Popular channels would hammer a single shard.
3. **Write amplification:** Every message write updates multiple indexes.
4. **Vertical scaling limits:** A single Postgres instance can only get so big.

For Discord's scale (trillions of messages, 60,000 writes/sec at peak), I need a distributed database designed for horizontal scaling.

Choosing Cassandra/ScyllaDB

I would choose a wide-column store like Cassandra or ScyllaDB (a C++ rewrite of Cassandra that's faster and more efficient). Here's why:

Strengths:

- **Horizontal scalability:** Add nodes to increase capacity and throughput
- **Tunable consistency:** Choose between strong and eventual consistency per query
- **Write-optimized:** Log-structured merge-tree (LSM) design handles high write throughput
- **No single point of failure:** Peer-to-peer architecture with replication

Tradeoffs:

- **No joins:** Have to denormalize data
- **Limited query flexibility:** Must design schema around query patterns
- **Eventual consistency challenges:** Need to handle conflicts

Discord actually migrated from Cassandra to ScyllaDB in 2022-2023 and saw dramatic improvements:

- **177 Cassandra nodes → 72 ScyllaDB nodes** (59% reduction)
- **P99 read latency:** 40-125ms → 15ms
- **Eliminated JVM garbage collection pauses** that were causing timeouts

The C++ rewrite eliminated the overhead and unpredictability of the JVM, making it a clear winner for this use case.

Schema Design: Partitioning by Channel

Here's my schema:

```
CREATE TABLE messages (
    channel_id BIGINT,
    bucket INT,
    message_id BIGINT,
    author_id BIGINT,
    content TEXT,
    attachments LIST<TEXT>,
    edited_timestamp TIMESTAMP,
    PRIMARY KEY ((channel_id, bucket), message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);
```

Let me break down each part:

Partition Key: `(channel_id, bucket)`

This determines which nodes store the data. All messages with the same `(channel_id, bucket)` live on the same set of nodes (3 nodes with replication factor 3).

Why include `bucket`? To prevent unbounded partition growth. If I used just `channel_id`, a popular channel active for years would have millions of messages in a single partition. Cassandra/Scylla partitions have practical size limits (~100MB-1GB recommended).

Bucketing strategy:

```
bucket = message_id // 10_000_000  # ~10M messages per bucket
```

For a channel with 50 million messages, this creates 5 buckets, distributing load across more nodes.

Clustering Key: `message_id DESC`

Within a partition, rows are sorted by `message_id` in descending order (newest first). This is perfect for the most common query: "give me the 50 most recent messages."

```
SELECT * FROM messages
WHERE channel_id = 789 AND bucket = 42
LIMIT 50;
```

This query reads from the start of the partition, so it is extremely fast.

Message ID Design (Snowflake)

I'm using Twitter's Snowflake ID format:

```
64 bits:
  41 bits: timestamp (milliseconds since epoch)
  10 bits: worker ID
  12 bits: sequence number
```

This gives me:

- **Globally unique IDs** without coordination
- **Time-ordered IDs** (higher ID = newer message)

- **69 years** of timestamps (until 2080-ish)
- **4096 IDs per millisecond** per worker

Because IDs are time-ordered, `message_id DESC` gives me chronological order for free.

Handling Pagination and History

Users need to scroll back through message history. Here's how I handle pagination:

Initial load (most recent 50 messages):

```
SELECT * FROM messages
WHERE channel_id = 789 AND bucket = 42
LIMIT 50;
```

Load older messages (pagination):

```
GET /channels/789/messages?before=123456789&limit=50
```

Backend query:

```
SELECT * FROM messages
WHERE channel_id = 789 AND bucket = 42
  AND message_id < 123456789
LIMIT 50;
```

Bucket boundary problem: What if the user scrolls back past the current bucket? I need to query the previous bucket:

```
def get_messages(channel_id, before_id=None, limit=50):
    if before_id:
        bucket = before_id // 10_000_000
    else:
        bucket = get_latest_bucket(channel_id)

    messages = query_messages(channel_id, bucket, before_id, limit)

    # If we didn't get enough messages, try previous bucket
    if len(messages) < limit and bucket > 0:
        remaining = limit - len(messages)
        older_messages = query_messages(channel_id, bucket - 1, None,
remaining)
        messages.extend(older_messages)

    return messages
```

This handles the bucket boundary transparently.

The Hot Partition Problem

Here's a real challenge: popular channels. Imagine a channel with 100,000 online members in a server for a popular game. Every time someone sends a message, thousands of users request message history simultaneously (as they open the channel or scroll up).

Even with ScyllaDB's 15ms P99 latency, if 10,000 users request messages at the same moment, that's 10,000 queries hitting the same 3 nodes (the replicas for that partition). Those nodes become a bottleneck.

Discord's solution: Request Coalescing Layer

They built a middleware layer in Rust that sits between the Gateway and the database. Here's how it works:

1. **Consistent routing:** Requests for `channel_id=789` always route to the same service instance (using consistent hashing on channel ID)
2. **Request deduplication:** If 10,000 users request the same messages (e.g., most recent 50 in channel 789), the service:
    - Sees that a query is already in-flight for that exact request

- Holds subsequent requests in memory
- When the database responds, broadcasts the result to all waiting requests
- **Result:** 10,000 requests → 1 database query

3. **Caching:** Frequently accessed messages (like the most recent 50 in a hot channel) are cached in memory with short TTL (10-30 seconds)

This architecture reduced database load by 90%+ for hot channels, independent of database performance improvements.

Handling Edits and Deletes

Message Edits:

I store the original message and track edits:

```
UPDATE messages
SET content = 'Hello world! (edited)', edited_timestamp = NOW()
WHERE channel_id = 789 AND bucket = 42 AND message_id = 123456789;
```

Optionally, I could store edit history in a separate table:

```
CREATE TABLE message_edits (
    message_id BIGINT,
    edit_timestamp TIMESTAMP,
    old_content TEXT,
    PRIMARY KEY (message_id, edit_timestamp)
);
```

Message Deletes:

Soft delete by setting a flag:

```
UPDATE messages
SET deleted = true, deleted_timestamp = NOW()
WHERE channel_id = 789 AND bucket = 42 AND message_id = 123456789;
```

Why soft delete? Compliance, audit logs, and abuse prevention. Admins might need to see deleted messages.

Tombstone Problem:

Cassandra/Scylla use tombstones to mark deleted data. Too many tombstones (millions of deletes in a partition) cause performance degradation: the database has to scan through tombstones to find live data.

Discord hit this hard with Cassandra. In channels with heavy message deletion, read latency spiked. ScyllaDB's more efficient compaction helped, but the real solution was:

- Periodic compaction tuning
- Limiting deletion rate
- Background cleanup jobs to fully remove old tombstones

Replication and Consistency

Replication Factor: 3

Every message is written to 3 nodes. If one node fails, two others have the data.

Consistency Level: Quorum

For writes:

```
QUORUM write = wait for 2 of 3 replicas to acknowledge
```

For reads:

```
QUORUM read = read from 2 of 3 replicas and return the newest version
```

This gives me strong consistency: if a write succeeds, subsequent reads will see it (assuming quorum on both).

Why not STRONG consistency (ALL replicas)?

If one replica is down or slow, the write would fail or timeout. Quorum provides a balance: tolerate one node failure while maintaining consistency.

Storage Growth and Capacity Planning

With 1 billion messages per day at 100 bytes each:

- **Daily growth:** 100 GB
- **Yearly growth:** 36.5 TB

With replication factor 3:

- **Yearly storage:** 109.5 TB

Over 5 years: ~550 TB of raw storage needed.

Media attachments (images, videos) are stored separately in object storage (S3) with CDN caching. Only URLs are stored in the message database, keeping message rows small.

Backup and Disaster Recovery

Continuous backups:

- ScyllaDB supports incremental backups to S3
- Point-in-time recovery within the last 7 days

Cross-region replication:

- For disaster recovery, replicate to a secondary region
- In case of full region failure, fail over to backup region

Retention policy:

- Messages stored indefinitely (unless deleted)
- Tombstones compacted after 10 days (configurable)

How Slack and Teams Differ

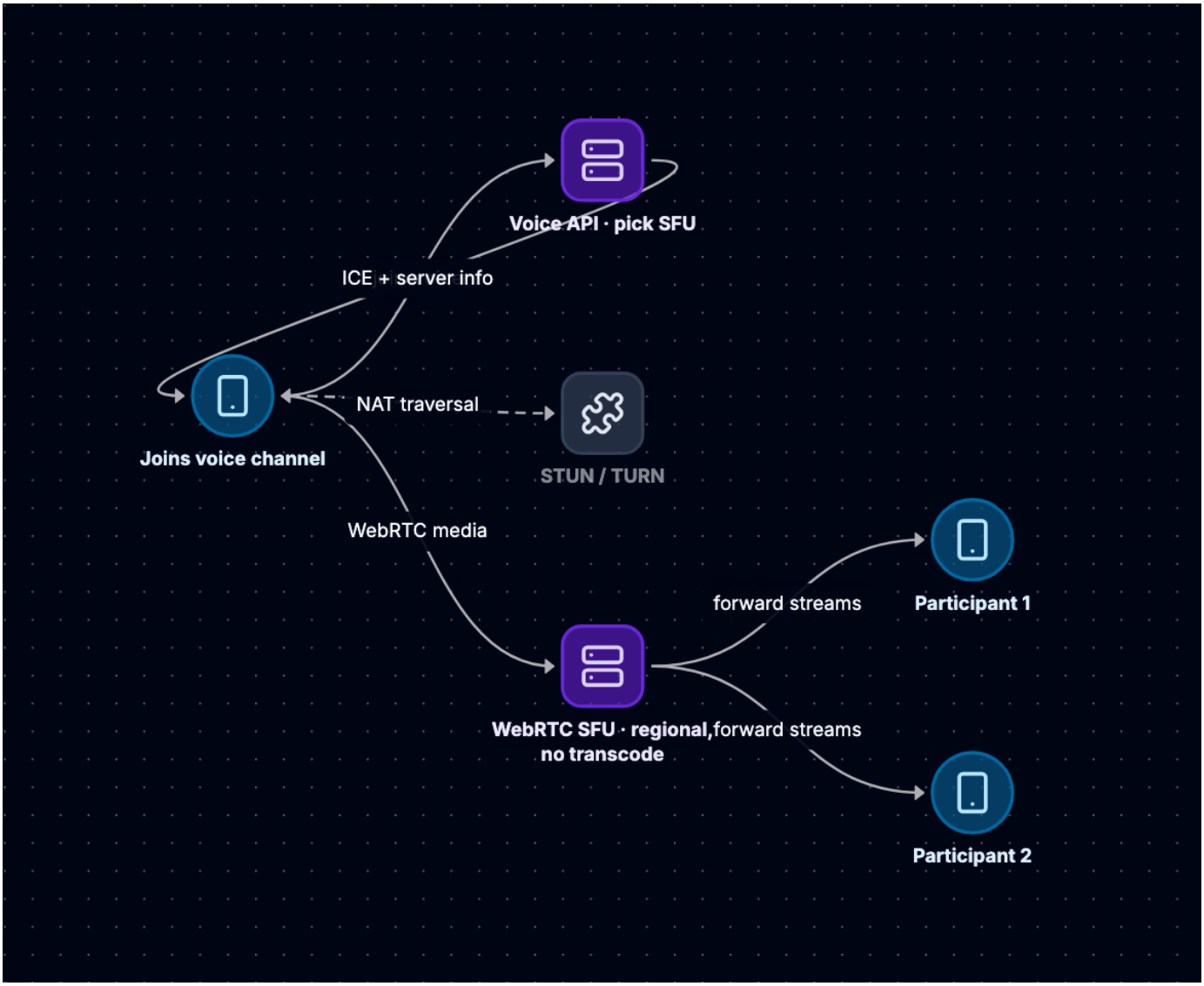
Slack uses a similar approach with sharded databases but has smaller message volumes per workspace. They also implement aggressive archiving: inactive workspaces are moved to cold storage (S3) and rehydrated on access.

Microsoft Teams stores messages in Exchange Online (mailbox infrastructure) and SharePoint for file attachments. This leverages existing Office 365 infrastructure but is less optimized for real-time chat workloads. Teams also has stricter retention policies driven by enterprise compliance needs (e.g., auto-delete after 30 days).

Discord's approach optimizes for high-volume, real-time access with indefinite retention, which fits their use case of persistent community servers.

Deep-Dive: Voice and Video Huddles (WebRTC / SFU)

Voice and video are where real-time communication gets really challenging. We're no longer dealing with kilobytes of text. We're routing megabits per second of audio and video streams with latency requirements measured in milliseconds. Let me walk through how I would build this.



The Latency Requirement

For natural conversation, latency must be under 150ms. Here's why:

- **Under 150ms:** Feels like normal conversation. People can interrupt naturally.
- **150-300ms:** Noticeable delay. Conversations feel awkward; people talk over each other.
- **Over 300ms:** Unusable for real-time conversation. Feels like walkie-talkies.

This means I have 150ms for:

- Audio capture and encoding on sender
- Network transmission

- Audio decoding and playback on receiver

If my server-side routing adds more than 50ms, I'm in trouble. This is why voice infrastructure must be regionally distributed: users in Tokyo can't route through a server in Virginia.

WebRTC: The Foundation

WebRTC (Web Real-Time Communication) is the standard for browser-based real-time audio/video. It handles:

- **Media capture:** Access to microphone and camera
- **Encoding:** Opus for audio (excellent quality at low bitrate), VP8/VP9/H.264 for video
- **NAT traversal:** STUN/TURN servers to establish connections through firewalls
- **Encryption:** DTLS-SRTP for secure media streams
- **Adaptive bitrate:** Adjusts quality based on network conditions

WebRTC is peer-to-peer by default, but for multi-user voice channels, I need a server in the middle.

Architecture Options: Mesh vs. MCU vs. SFU

1. Mesh (Peer-to-Peer)

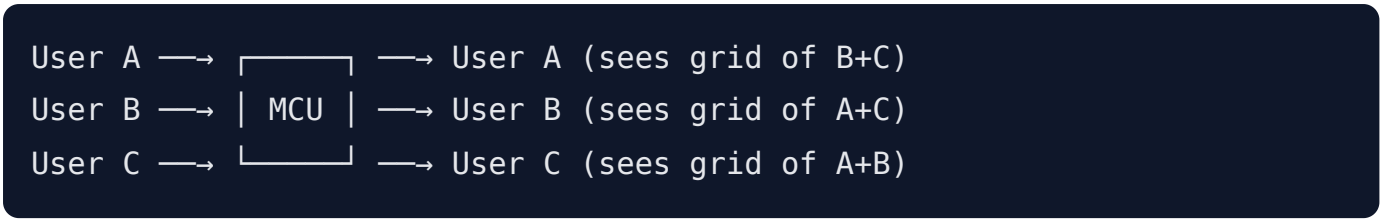
Each participant sends their stream directly to every other participant.



Pros: No server infrastructure needed, lowest latency
Cons: Doesn't scale beyond 3-4 participants. For N users, each uploads N-1 streams. A 10-person call means each user uploads 9 video streams, which is impossible on typical home internet.

2. MCU (Multipoint Control Unit)

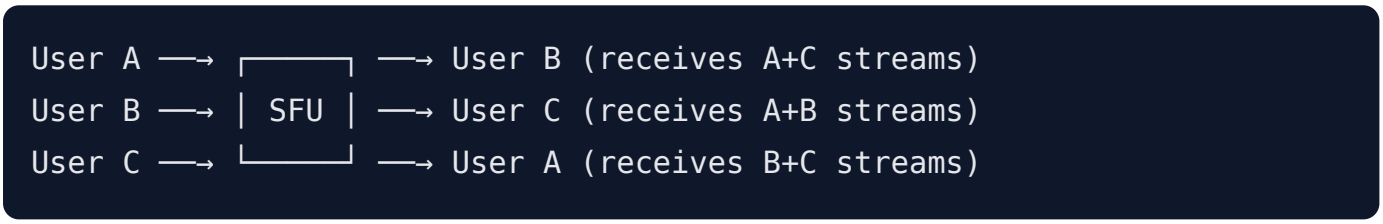
Server receives all streams, decodes them, composites them into a single video (like a grid of faces), re-encodes, and sends the composite to each participant.



Pros: Each user only uploads/downloads 1 stream
Cons: Server must decode, composite, and re-encode all streams, which is extremely CPU intensive. Doesn't scale well. Everyone sees the same layout.

3. SFU (Selective Forwarding Unit)

Server receives streams from each participant and forwards them to others *without decoding or re-encoding*.



Pros: Server just forwards packets, so it needs minimal CPU. Scales to dozens of participants. Clients can choose which streams to render (e.g., only show active speaker).
Cons: Each user downloads N-1 streams (bandwidth intensive on receiver).

My choice: SFU

For Discord's use case (voice channels with 2-25 typical participants, up to 50), SFU is the sweet spot. It scales far better than MCU, and the download bandwidth requirement is manageable with adaptive bitrate.

Discord uses SFU architecture, as do Zoom, Google Meet, and most modern video conferencing platforms.

SFU Media Server Design

Here's my SFU architecture:

Components:

1. **Media Server (SFU):**
   - Receives RTP/RTCP packets from clients
   - Forwards packets to other participants
   - Handles STUN/TURN for NAT traversal

- Monitors packet loss and adapts quality

2. **Signaling Server:**
   - Coordinates WebRTC connection setup (SDP offer/answer exchange)
   - Tells clients which SFU to connect to
   - Handles join/leave events
3. **TURN Server:**
   - Relays traffic when direct connection fails (strict firewalls/NATs)
   - Fallback only, and adds latency

Connection Flow:

1. **User joins voice channel:**

```
POST /channels/456/join-voice
```

2. **Signaling server responds:**

```
{
  "sfu_address": "sfu-us-east-1.discord.gg:50000",
  "ice_servers": [
    { "urls": "stun:stun.discord.gg:3478" },
    { "urls": "turn:turn.discord.gg:3478", "username": "...",
"credential": "..." }
  ],
  "session_id": "abc123"
}
```

3. **Client initiates WebRTC connection:**
   - Creates `RTCPeerConnection`
   - Generates SDP offer (describes supported codecs, bitrates)
   - Sends offer to signaling server
4. **SFU responds with SDP answer:**
   - Describes which codecs it supports

- Includes ICE candidates (IP addresses/ports to try)

5. **ICE negotiation:**
   - Client and SFU exchange ICE candidates
   - Try direct connection (best case)
   - Fall back to TURN relay if direct fails
6. **Media flows:**
   - Client sends RTP packets with Opus-encoded audio to SFU
   - SFU forwards to all other participants in the channel

Handling Multiple Participants

When 10 users are in a voice channel:

- Each user sends 1 audio stream to the SFU
- SFU forwards each stream to the other 9 users
- Each user receives 9 audio streams
- Client mixes the 9 streams locally (audio mixing is cheap)

For video, it's trickier. Receiving 9 video streams is bandwidth-intensive. Solutions:

Simulcast:
Each sender uploads multiple resolutions (e.g., 1080p, 720p, 480p, 180p). SFU forwards the appropriate resolution to each receiver based on their bandwidth and screen size.

Active Speaker Detection:
SFU detects who's speaking (based on audio levels) and sends high-quality video for the active speaker, lower quality for others.

Regional Distribution

To achieve sub-150ms latency globally, I need SFU servers in multiple regions:

- **North America:** US East, US West
- **Europe:** London, Frankfurt
- **Asia:** Tokyo, Singapore
- **South America:** São Paulo
- **Oceania:** Sydney

When a user joins a voice channel, the signaling server picks the closest SFU based on:

- User's geographic location (IP geolocation)
- Current SFU load
- Existing participants' locations (prefer SFU closest to majority)

For a voice channel with users in New York and London, the SFU might be placed in New York (closer to majority) or London (depending on distribution). Users farther from the SFU have higher latency but still under 150ms with good routing.

Scaling Voice Channels

Each SFU server has finite capacity:

- **Bandwidth:** If each user sends 64 kbps audio, 50 users = 3.2 Mbps inbound, 160 Mbps outbound (forwarding to 49 others). A 10 Gbps NIC can handle ~60 such channels.
- **CPU:** Packet forwarding is lightweight, but DTLS encryption/decryption and RTP processing add up. A 16-core server can handle ~500 participants across multiple channels.

When a channel exceeds a single SFU's capacity, I can:

- **Split the channel:** Move some users to a second SFU, with SFUs forwarding streams to each other (cascading SFUs)
- **Limit channel size:** Discord caps voice channels at 50-100 users

For massive events (e.g., 10,000 listeners), I'd switch to a broadcast model: one speaker, many listeners. This is a different architecture (CDN-based streaming with 5-10 second delay).

Audio Quality and Codec Choice

Opus codec is the gold standard for voice:

- **Bitrate range:** 6 kbps (narrow-band) to 510 kbps (full-band stereo)
- **Typical voice:** 32-64 kbps (excellent quality)
- **Low latency:** 20ms frame size
- **Adaptive:** Adjusts to network conditions

For video, I'd use **VP8** or **H.264**:

- **VP8:** Open-source, good quality, wide browser support
- **H.264:** Better hardware encoding/decoding, slightly better quality
- **Bitrate:** 500 kbps - 2 Mbps depending on resolution

Handling Network Issues

Real-world networks are unreliable. I need to handle:

Packet Loss:

- **FEC (Forward Error Correction):** Send redundant data so lost packets can be reconstructed
- **NACK (Negative Acknowledgment):** Receiver requests retransmission of lost packets
- **Opus built-in PLC (Packet Loss Concealment):** Synthesizes missing audio frames

Jitter (Variable Latency):

- **Jitter buffer:** Hold packets briefly to smooth out arrival times
- **Adaptive buffer size:** Grow buffer if jitter increases, shrink if network stabilizes

Bandwidth Fluctuations:

- **REMB (Receiver Estimated Maximum Bitrate):** Receiver tells sender how much bandwidth is available
- **Sender reduces bitrate** (lower resolution or frame rate) if network is congested

Monitoring Voice Quality

Critical metrics:

- **Packet loss rate:** Should be < 1%. Above 5% = poor quality.
- **Latency (RTT):** Round-trip time. Should be < 100ms.
- **Jitter:** Variation in latency. Should be < 30ms.
- **MOS (Mean Opinion Score):** Subjective quality score (1-5). Target > 4.0.

I'd also track:

- **Call setup success rate:** % of users who successfully connect
- **Call drop rate:** % of calls that disconnect unexpectedly
- **Server CPU/bandwidth usage:** Capacity planning

How Slack and Teams Differ

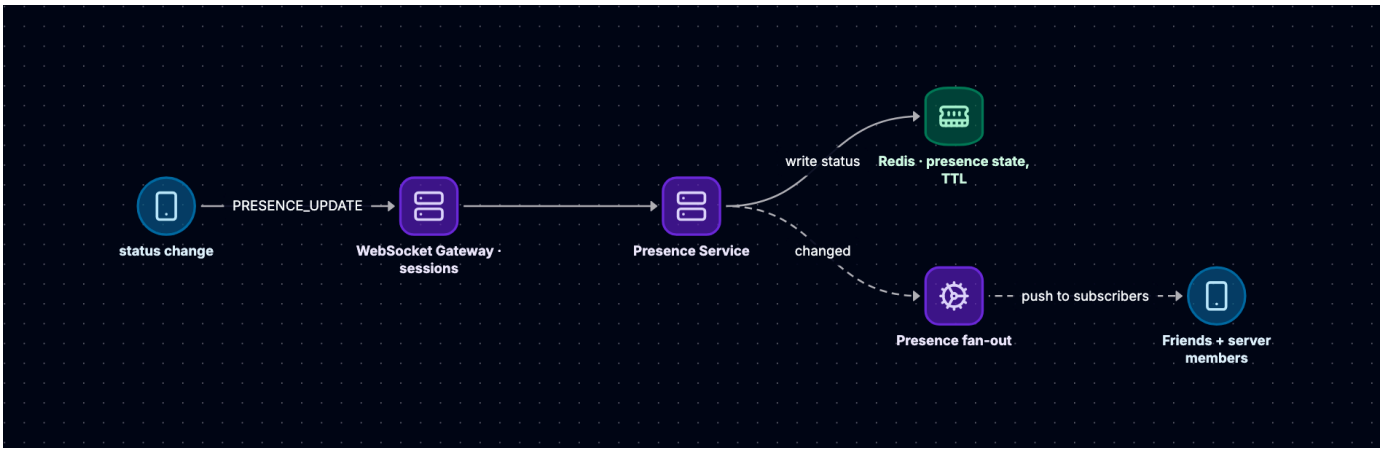
Slack uses a third-party provider (formerly Screenhero, now integrated) for voice/video. Their huddles are lightweight (audio-first) and optimized for small teams (2-10 people).

Microsoft Teams uses Azure Communication Services with a hybrid MCU/SFU approach. For large meetings (100+ participants), they use MCU for bandwidth efficiency. They also have tight integration with traditional telephony (PSTN calling) via SIP trunking.

Discord's SFU-based architecture is optimized for gaming communities where low latency and high quality are critical, and typical voice channels have 5-20 participants.

Deep-Dive: Presence and Real-Time State

Presence, knowing who's online, who's typing, who's in a voice channel, is a defining feature of real-time communication platforms. It's also one of the hardest problems to scale. Let me walk through the challenges and solutions.



The N-Squared Broadcasting Problem

Here's the nightmare scenario: a server with 10,000 members. If 1% of members change their status every 10 seconds (going online, offline, idle, starting to type):

- **100 status updates per 10 seconds** = 10 updates/second
- Each update must be broadcast to all 10,000 members
- **10 updates/sec × 10,000 recipients** = **100,000 messages/second** for a single server

Now scale this to 1,000 active large servers:

- **100,000 messages/sec × 1,000 servers** = **100 million messages/second** platform-wide

This is the N-squared problem: presence updates scale with the product of users and their friends/server members.

Discord's Presence Crisis

Discord hit this problem hard in their early days. Their naive implementation:

1. User status changes (online → idle)
2. Gateway server looks up all servers the user is in
3. For each server, broadcast `PRESENCE_UPDATE` to all members
4. Each broadcast allocated a new JSON object in memory

Result: Gateway servers were allocating 50 MB/sec of heap memory just for presence updates. The JVM's young generation garbage collector ran every 2-3 seconds, causing 100-500ms pauses. During GC pauses, all threads froze, so no messages could be delivered.

With 10 concurrent broadcasts, the thread pool had 100,000 tasks queued, leading to thread starvation and timeouts.

Zero-Allocation Solution

Discord's fix was brilliant: eliminate allocations entirely.

1. Immutable Shared Records

Instead of serializing the presence update for each recipient:

```
// BAD: Allocates 10,000 strings
for (User user : server.members) {
    String json = serializePresenceUpdate(update);  // New allocation!
    user.sendMessage(json);
}
```

They serialize once and share:

```
// GOOD: Allocate once, share read-only views
ByteBuffer sharedBuffer = serializePresenceUpdate(update);  // One
allocation
for (User user : server.members) {
    user.sendMessage(sharedBuffer.asReadOnlyBuffer());  // No allocation
}
```

Impact: 50 MB/sec → 500 KB/sec allocation rate (99% reduction).

2. Lock-Free Registry

To look up "which users are in this server," they used:

```
ConcurrentHashMap<ServerId, CopyOnWriteArrayList<User>>
```

- **Reads are lock-free:** Broadcaster threads never block
- **Writes are rare:** Users join/leave servers infrequently
- **Copy-on-write:** When a user joins, the list is copied (expensive), but reads are fast (critical)

3. Heartbeat Protocol

Instead of broadcasting every status change, clients send heartbeats:

```
Client → Server: { "op": 1, "d": null }  (every 40 seconds)
```

If the server doesn't receive a heartbeat within 40s + 10s grace period, it marks the user offline. This reduces presence update frequency.

Presence Scope: Friends vs. Servers

I need to distinguish between two types of presence:

1. Friend Presence:

- User A is friends with User B
- When A goes online, B should see it immediately
- Typically, users have 10-100 friends

2. Server Presence:

- User A is in a server with 10,000 members
- When A goes online, all 10,000 should see it (eventually)
- But do they need to see it *immediately*?

My approach: Different update frequencies.

- **Friend presence:** Real-time (< 1 second)
- **Server presence:** Batched (every 30-60 seconds) or lazy-loaded (only when viewing member list)

This dramatically reduces fan-out. If I only broadcast friend presence in real-time, and a user has 50 friends, that's 50 messages per status change, which is manageable.

Presence Data Model

User Presence State:

```
{
  "user_id": 1001,
  "status": "online",  // online, idle, dnd, offline
  "activities": [
    {
      "name": "Playing Elden Ring",
      "type": "game",
      "details": "Exploring Limgrave"
    }
  ],
  "client_status": {
    "desktop": "online",
    "mobile": "idle"
  }
}
```

Storage:

- Presence is ephemeral, with no need to persist to disk
- Store in Redis or in-memory on Gateway servers
- Key: `user:{user_id}:presence`
- TTL: 5 minutes (refreshed by heartbeats)

Presence Fan-Out Strategy

When User A's status changes:

Step 1: Update presence store

```
SET user:1001:presence '{"status":"online",...}'
EXPIRE user:1001:presence 300
```

Step 2: Publish to Kafka

```
{
  "topic": "presence",
  "key": "user:1001",
  "value": {
    "user_id": 1001,
    "status": "online",
    "timestamp": "2024-01-15T10:30:00Z"
  }
}
```

Step 3: Gateway servers consume and fan out

Each Gateway server:

1. Receives presence update from Kafka
2. Looks up User A's friends who are connected to *this* Gateway
3. Sends `PRESENCE_UPDATE` to those friends via WebSocket

Step 4: Lazy-load for servers

For server members (not friends), I don't push updates immediately. Instead:

- When User B opens the member list for a server, they request:

  ```
  GET /servers/789/members?with_presence=true
  ```

- API fetches presence for all members from Redis
- Client caches this and subscribes to updates for visible members only

Typing Indicators

Typing indicators are even more ephemeral than presence. When User A starts typing in a channel:

Client sends:

```
{ "op": 8, "d": { "channel_id": 789 } }
```

Gateway broadcasts to channel members:

```
{
  "op": 0,
  "t": "TYPING_START",
  "d": {
    "channel_id": 789,
    "user_id": 1001,
    "timestamp": "2024-01-15T10:30:00Z"
  }
}
```

Key optimizations:

1. **Rate limiting:** Client can only send typing indicator once per 5 seconds
2. **No persistence:** Typing state is never written to database
3. **Timeout:** If no message is sent within 10 seconds, client stops showing indicator
4. **Batching:** If 5 people are typing, Gateway batches into one event: `"users_typing": [1001, 1002, 1003, 1004, 1005]`

Voice Channel Occupancy

Knowing who's in a voice channel is critical presence information. When User A joins a voice channel:

Client sends:

```
POST /channels/456/join-voice
```

API server:

1. Updates voice state in Redis:

   ```
   SADD voice_channel:456:members 1001
   SET user:1001:voice_state
   '{"channel_id":456,"self_mute":false,"self_deaf":false}'
   ```

2. Publishes to Kafka:

```
{
  "topic": "voice_state",
  "value": {
    "user_id": 1001,
    "channel_id": 456,
    "action": "join"
  }
}
```

3. Gateway servers broadcast to server members:

```
{
  "op": 0,
  "t": "VOICE_STATE_UPDATE",
  "d": {
    "user_id": 1001,
    "channel_id": 456,
    "self_mute": false,
    "self_deaf": false
  }
}
```

When viewing a server:

- Client receives list of all members currently in voice channels
- UI shows user avatars in voice channel list
- Updates in real-time as people join/leave

Gateway Session Management

Each Gateway server maintains session state for connected users:

```
type Session struct {
    UserID      int64
    SessionID   string
    Connection  *websocket.Conn
    Heartbeat   time.Time
    Servers     []int64  // Servers user is in
    Friends     []int64  // User's friends
}

// In-memory map
sessions := make(map[int64]*Session)  // user_id -> session
```

When a presence update arrives, the Gateway:

1. Checks if any friends of the updating user are connected to this Gateway
2. Sends the update to those sessions

This requires the Gateway to know each user's friend list. Options:

Option A: Cache friend lists in Gateway

- On connect, fetch user's friends from database
- Cache in Gateway memory
- Invalidate on friend add/remove events

Option B: Query on each update

- Gateway queries Redis: `SMEMBERS user:1001:friends`
- Higher latency but always fresh

I'd choose **Option A** with cache invalidation. Friend lists change infrequently, so caching is effective.

Handling Presence at Scale

With 10 million concurrent users:

Presence updates per second:

- Assume 1% of users change status every minute
- 10,000,000 × 0.01 / 60 = **1,667 status changes/second**

Fan-out per update:

- Average 50 friends per user
- 1,667 × 50 = **83,350 presence messages/second**

Gateway load:

- 200 Gateway servers
- 83,350 / 200 = **~417 messages/sec per Gateway**

This is manageable with the zero-allocation approach. Each Gateway handles 417 outbound messages/sec with minimal CPU and memory overhead.

Monitoring Presence Health

Key metrics:

- **Presence update latency:** Time from status change to friend receiving update. Target < 1 second.
- **Heartbeat miss rate:** % of users missing heartbeats (indicates network issues or client crashes)
- **Presence fan-out rate:** Messages/sec per Gateway
- **Redis latency:** Presence queries should be < 5ms

If presence latency spikes above 5 seconds, users complain that friends appear offline when they're actually online, a critical UX issue.

How Slack and Teams Differ

Slack has simpler presence requirements due to smaller workspace sizes. They use a similar Redis-based approach but don't face the same N-squared problem because workspaces rarely exceed 1,000 active users.

Microsoft Teams batches presence updates aggressively: status changes can take 5+ minutes to propagate. This is acceptable in enterprise settings where presence is less critical than in gaming communities. Teams also integrates with Exchange for calendar-based presence (e.g., "In a meeting").

Discord's real-time presence is a competitive advantage for gaming communities where knowing "who's online right now" is essential for spontaneous voice chats.

Conclusion: Bringing It All Together

Designing a real-time chat and voice platform like Discord is a masterclass in distributed systems engineering. Every component, from the WebSocket Gateway to the message database to the SFU media servers, must be carefully architected to handle massive scale while maintaining the low latency that makes real-time communication feel natural.

Here's what I've learned from walking through this design:

1. Separate stateful from stateless. The WebSocket Gateway holds persistent connections (stateful) while the REST API handles business logic (stateless). This separation makes the system easier to scale and deploy.

2. Request coalescing is critical for hot spots. Even with a fast database like ScyllaDB (15ms P99 latency), popular channels can overwhelm the system. Discord's Rust-based request coalescing layer reduced database load by 90%+ by merging identical requests.

3. Zero-allocation broadcasting scales presence. The N-squared problem of broadcasting presence updates to thousands of users is solved by eliminating memory allocations: serializing once and sharing read-only byte buffers. This reduced Discord's allocation rate by 99%.

4. SFU architecture wins for voice/video. Selective Forwarding Units strike the perfect balance between scalability and quality, forwarding media streams without the CPU cost of transcoding. Regional distribution keeps latency under 150ms for natural conversation.

5. Partitioning by channel is the right data model. Storing messages in ScyllaDB partitioned by `(channel_id, bucket)` with clustering by `message_id DESC` optimizes for the most common query pattern: "give me recent messages in this channel."

6. Monitor everything in real-time. Real-time systems fail in real-time. Tracking metrics like message delivery latency, presence update latency, packet loss, and database P99 latency is essential for catching problems before users notice.

The numbers tell the story: **10 million concurrent users**, **60,000 peak messages per second**, **2.5 million concurrent voice users**, **trillions of messages stored**, all delivered with **sub-second message latency** and **sub-150ms voice latency**. Achieving this requires careful attention to every layer of the stack, from choosing the right database (ScyllaDB's 59% node reduction) to optimizing memory allocation patterns (99% reduction in GC pressure).

If I were building this system from scratch today, I'd start with the architecture I've outlined here: a hybrid REST/WebSocket API, ScyllaDB for message storage, regional SFU servers for voice/video, and Redis for ephemeral presence state. But I'd also plan for iteration, because Discord didn't get everything right on the first try. They migrated from Cassandra to

ScyllaDB, rewrote hot paths in Rust, and continuously optimized based on real-world usage patterns.

The beauty of distributed systems is that there's always another bottleneck to optimize, another nine of availability to chase, another region to expand into. But with the right architectural foundation (the one I've laid out in this post), you can scale from hundreds of users to hundreds of millions while keeping the magic of real-time communication alive.